ChatBot Overview

First Draft

(Drafted keeping Ops360 in mind)

TABLE OF CONTENTS

1. Objective 1
2. Functional Requirements 1
3. Proposed Architecture for ChatBot (potential) 2
4. Knowledge Acquisition: 3
5. Rough flow 3
6. Integration with existing Chat App 3
7. Timeline 4

1. Objective

For an organization, there exists a web portal (and an app). Inside the web portal there is a Chat App feature. The purpose of this Chat App is to establish communication between the two users. We wish to add an option of chatting with a bot as well.
In the organization, there are multiple departments like Finance, ATM, Administration, etc. For each department, there will be a department-specific ChatBot. Depending on the department to which the user belongs, that specific Chat Bot will get triggered.
So, if a user has a query related to their department, instead of communicating with an expert (user), they can choose to communicate with the bot.

We will ensure that this is:

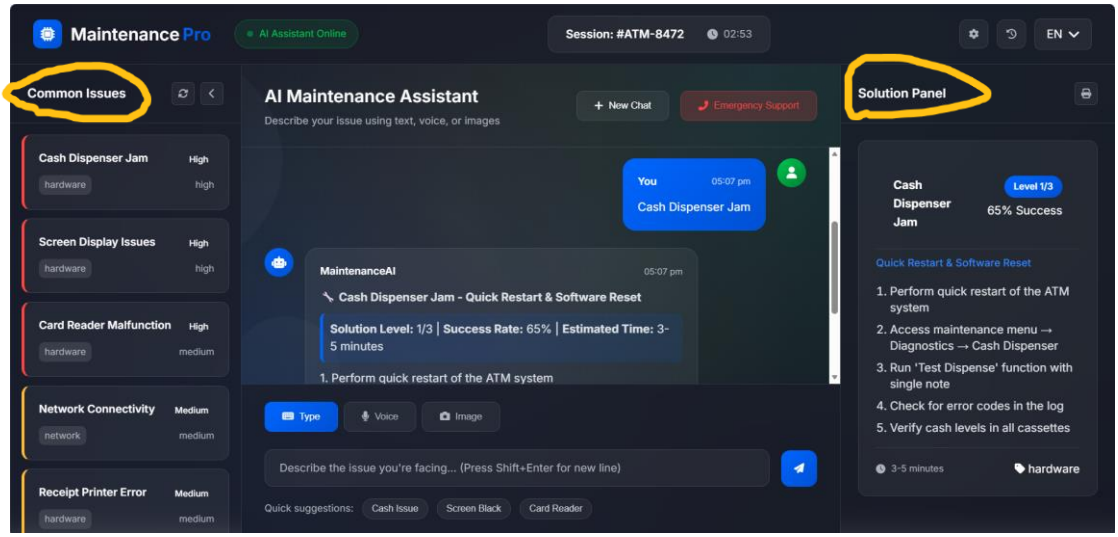
- On premises
- Offline: no internet connection required
- Domain-Specific
- Scalable-The same architecture can be deployed for any department (Finance, HR,
- etc.) by simply changing the underlying knowledge base.

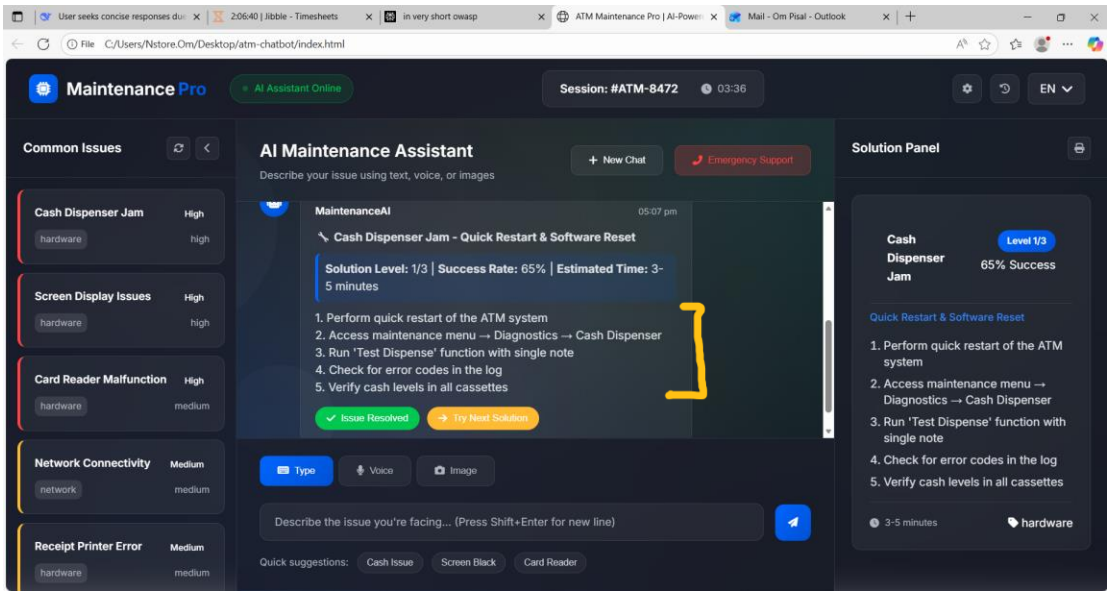
2. Functional Requirements

1. **ChatBot for each department (multiple ChatBots)**
   Each department will have its own ChatBot. This will prevent the bot from giving generalized solutions and will make it specialized for a domain.
   Each ChatBot will be stored on the server having a departmentId of the department it belongs to.

2. **JSON file for Common Issues**
   A simple JSON file maps frequent user queries directly to pre-approved solutions for instant answers.
   This will be a section on the side (a), clicking on which will pop out the simple solution in the chat (b).

   Sample image:

a. 

b. 

3. **JSON File for SMEs**

For situations where Bot is not too sure about the answer (confidence < 0.7), bot can fetch and present the contact details of the Subject Matter Experts.

These contact details will be in the JSON file and depending on the department/domain of the user, a specific SME information can be fetched.

3. Proposed Architecture for ChatBot (potential)

All components are open-source, ensuring no licensing costs and full offline capability.

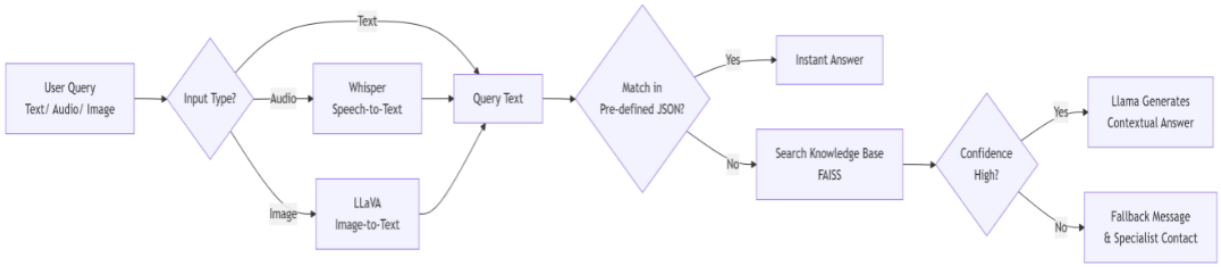
a. **Core AI Framework:** LangChain
b. **Large Language Model (LLM)**: Llama 3 (open-source model for generating textual responses)
c. **Multimodal Understanding**:
   - Images: LLaVA (Analyzes images and generates text descriptions)
   - Audio: Whisper (Converts speech to text with high accuracy)
d. **Knowledge Base**: FAISS (An efficient vector database for instant information retrieval)
e. **Backend & Integration:**
   - API Server: Python with FastAPI
   - Real-Time Communication: WebSockets

4. Knowledge Acquisition:

This chatbot will be able to answer the questions of the user with solutions designed from being trained on user manuals or other sources of documentation (data) provided by the departments.

This data can be both structured (xlsx, csv, database, etc.) and unstructured (image, pdf, etc.).

5. Rough flow



6. Integration with existing ChatApp

Currently, there are two screens.

First Screen: Chat App-- where the users see a list of their existing conversations (with other users).

Second Screen: Options:

a. Start a New Chat with the assigned user

b. Chat with Bot

In the Chat App, a user token is passed every time a user opens it. The token is decoded to extract information like name, departmentName, departmentId, email etc. There is a button that the user clicks which gives them an option to either Start a New Chat or Chat with Bot.

User clicks on ‘Chat with Bot’. Their department Id is sent to the server corresponding to which a specific Chat Bot will be triggered and will be in running state. The user opens a fresh chat, and the bot is now running. Note: Server is hit only once to link the specific bot to the chat. Then user continues chatting to the bot.

7. Timeline

Work begins on 27th November, 2025 (Thursday). The timeline below is for one bot. We can work on multiple bots in parallel.

Week	**Work**	**Description**
Week 0 (Thur, 27 Nov)	Brainstorm	Finalize on the Tech Stack

Week 1 (Mon, 1 Dec)	Data EDA	Collect data, add data preparation steps and load it.
Week 2 (Mon, 8 Dec)	Embedding Layer and Vector Store	Creating chunks from the document (or other data source) and creating embeddings. Set up a Vector DB and store it in there.
Week 3 (Mon, 15 Dec)	Model Train	Building Classification Layer
Week 4 (Mon, 22 Dec)- Holiday week	Test Model	Testing on Unseen Data
Week 5 (Mon, 29 Dec)- Holiday Week	Test Model	Testing and fine-tuning
Week 6 (Mon, 5 Jan)	Final Model	Hosting it on the server, integrating APIs
Week 7 (Mon, 12 Jan)	FrontEnd/UI	Designing and developing UI
Week 8 (Mon, 19 Jan)	Add websockets and connecting with node server	For real-time messaging
Week 8 (Mon, 19 Jan)	Integration of all components	Integrate APIs to call specific models stored on the server begin and sending and receiving messages
Week 9 (Tue, 27 Jan)- Holiday Week	Testing	Integration Testing
Week 10 (Mon, 2 Feb)	Deployment	

NOTE: This timeline is only for textual prompts and outputs. Yet to add how the model will behave with images and audio.

And this is subject to change in the upcoming discussion meetings.